EXECUTIVE PRESENTATION DECK

Enterprise Revenue & Node Reward Architecture

How a corporate storage payment actually flows through the protocol to a node operator's wallet.

Document INAYA-REV-2026-V2 · Classification Confidential · August 2026

SECTION 01

Revenue Router — Payment Settlement

When an enterprise customer purchases a Corporate Reserve plan, they approve USDT and call RevenueRouter.processCorporateInvoice(usdtAmount) — a real, standalone on-chain transaction.

ALLOCATION	PURPOSE
[VERIFY]%	Node Reward Escrow
[VERIFY]%	Company Treasury
[VERIFY]%	Team & Platform Maintenance

What changed from the prior edition. The revenue-split percentages are stated as [VERIFY] rather than restated as fact, since RevenueRouter's Solidity source isn't tracked in this repository — only its deployed address and an inline ABI fragment are. Confirm the current split with the founders before quoting it in an institutional setting.

SECTION 02

Corporate Escrow — Funding The Operator Pool

After the router transaction confirms, the client separately approves USDT and calls InayaCorporateEscrow.createEscrow(corporate, node, cogsAmount) — a second, independent transaction, not an automatic chain reaction triggered by the router itself.

- Creates a unique, auditable escrow record on-chain, keyed by an incrementing scheduleId.
- Locks the allocated funds for a fixed 12-month vesting period.
- Releases via releaseMonthlyPayout(scheduleId) — publicly callable by anyone once a release is due, not restricted to the protocol or the node operator.

> **Correction from prior edition.** Earlier material described escrow creation as automatic — "no manual approval step," happening as part of the same settlement flow as the router transaction. It is, in fact, a second explicit transaction in the client's checkout code. The distinction matters for anyone reasoning about transaction atomicity or failure modes (e.g. what happens if a user's wallet rejects the second approval after the first transaction already succeeded).

SECTION 03

Vesting Mechanics

Rather than paying operators immediately, the escrow distributes rewards gradually — protecting protocol liquidity while giving operators a predictable, recurring income stream.

1. **1.** Annual allocation locked in InayaCorporateEscrow.
2. **2.** 12 fixed monthly releases, each unlocked once its due date passes.
3. **3.** Anyone (not just the protocol) can call releaseMonthlyPayout() to trigger a due release.

> [VERIFY] a specific worked-example dollar figure with the founders before publishing one — a prior edition used a placeholder "5,265 USDT/year" example that doesn't map to the real 250/500/1000 TB tier pricing (13,500 / 27,000 / 54,000 USDT/yr) and should not be reused without recalculating from real numbers.

SECTION 04

Node Registry — Performance Tracking

InayaNodeRegistry continuously maintains each node's operational record: capacity, reputation score, historical uptime, active status, stake balance, and assigned workloads.

> **Trust-model honesty, on the record.** The contract's own header comment is explicit: node metrics are coordinator-verified — an authorized verifier wallet attests uptime/capacity off-chain — not cryptographically proven. This document states that plainly rather than implying stronger guarantees than the contract actually makes.

SECTION 05

Proof Registry — Storage Verification

At intervals, the protocol issues cryptographic storage challenges. Each node must prove it still holds its assigned encrypted shard.

- Merkle proof validity — the submitted proof correctly resolves against the expected root.
- Requested chunk availability — the challenged data segment is actually retrievable.
- Challenge response deadline — the proof must land within the required window.

> Today, verifyChunkProof() is backend-checked (onlyOwner) — the contract's own header comment documents a planned future path to make this permissionless and stake-slashing-backed. Not yet built; stated as a roadmap item, not current behavior.

SECTION 06

Reward Authorization Flow

A node satisfying both operational requirements and Proof-of-Storage verification becomes eligible for that period's reward release — nodes that fail verification simply don't receive that period's payout; repeated failures reduce reputation and may eventually result in removal.

SECTION 07

Swarm Reserve — Tokenomics

Emissions from the Swarm Reserve (40% of total $INAYA supply) are uptime-gated to prevent mercenary liquidity dumping.

TIER	QUARTERLY UPTIME	MONTHLY CAP	RATING
Tier 1 (Elite)	98.0–100%	30 $INAYA	Enterprise-grade
Tier 2 (Standard)	95.0–97.9%	20 $INAYA	Commercial-grade
Tier 3 (Baseline)	90.0–94.9%	10 $INAYA	Minimum viable
Ineligible	< 90.0%	0 $INAYA	Slash / reputation penalty

[VERIFY] these specific thresholds and caps with the founders — this is a tokenomics design decision, not something currently enforced directly by InayaNodeRegistry's on-chain commission logic (which is confirmed in code). Confirm whichever system actually governs $INAYA emission before restating these numbers as current fact.

SECTION 08

Why This Architecture Matters

- Enterprises pay in USDT through a familiar, predictable billing model.
- Node operators earn predictable recurring income, but only by continuously proving storage.
- The protocol preserves liquidity through escrowed vesting rather than immediate payouts.
- Revenue distribution is transparent and on-chain, auditable by anyone.

> This ties operator compensation directly to verified network performance — real, on-chain, but via two client-initiated transactions rather than a single atomic cascade, as corrected above.